

BANCO DE DADOS RELACIONAL

Stored Procedure

Professora:

Lucineide Pimenta

Recapitulando a aula anterior



- ❑ O que é uma VIEW?
- ❑ Qual a diferença entre SQL salvo e VIEW?
- ❑ Quando usar VIEW?
- ❑ Vantagem de segurança ao usar VIEW?

BANCO DE DADOS RELACIONAL

Correção dos exercícios da aula anterior

Exercício 1: Reservatórios com média de temperatura maior que a média geral

- ❑ **Enunciado resumido:** encontre os reservatórios cuja média de Temperatura (> por reservatório) é maior que a média global de Temperatura (todos os registros).

- ❑ -- Solução: média por reservatorio > média global

```
SELECT r.nome AS reservatorio,  
       AVG(s.valor) AS media_temperatura  
FROM serie_temporal s  
JOIN parametro p ON s.id_parametro = p.id_parametro  
JOIN reservatorio r ON s.id_reservatorio = r.id_reservatorio  
WHERE p.nome_parametro = 'Temperatura'  
GROUP BY r.id_reservatorio, r.nome
```

```
HAVING AVG(s.valor) > (  
    SELECT AVG(s2.valor)  
    FROM serie_temporal s2  
    JOIN parametro p2 ON s2.id_parametro =  
    p2.id_parametro  
    WHERE p2.nome_parametro = 'Temperatura'  
)  
ORDER BY media_temperatura DESC;
```

Exercício 1: Reservatórios com média de temperatura maior que a média geral

- ❑ **Explicação:**
- ❑ **WHERE p.nome_parametro = 'Temperatura'** filtra apenas registros de temperatura.
- ❑ **GROUP BY r.id_reservatorio, r.nome** agrupa por reservatório.
- ❑ **HAVING AVG(s.valor) > (subconsulta)** compara a média do grupo com a média global (subconsulta).
- ❑ **ORDER BY** para mostrar os maiores primeiros.
- ❑ **Observações:** se não houver registros de temperatura para um reservatório, ele não aparece. Se quiser incluir reservatórios sem medições (com NULL), precisaríamos LEFT JOIN em **serie_temporal** e tratar NULLs, mas aqui faz sentido excluir quem não tem dados.

Exercício 2: Top 5 maiores valores de turbidez por reservatório

- ❑ **Enunciado resumido:** para cada reservatório, listar as 5 maiores medições de Turbidez (valor + data).
- ❑ -- Solução usando ROW_NUMBER() para top-N por partição

```
SELECT nome_reservatorio, id_serie, valor, data_hora, rn  
FROM (
```

```
    SELECT r.nome AS nome_reservatorio,  
           s.id_serie,  
           s.valor,  
           s.data_hora,  
           ROW_NUMBER() OVER (  
               PARTITION BY s.id_reservatorio  
               ORDER BY s.valor DESC  
           ) AS rn
```

```
FROM serie_temporal s
```

```
    JOIN parametro p ON s.id_parametro =  
p.id_parametro
```

```
    JOIN reservatorio r ON s.id_reservatorio =  
r.id_reservatorio
```

```
    WHERE p.nome_parametro = 'Turbidez'
```

```
) t
```

```
WHERE rn <= 5
```

```
ORDER BY nome_reservatorio, rn;
```

Exercício 2: Top 5 maiores valores de turbidez por reservatório

- ❑ **Explicação:**
- ❑ **ROW_NUMBER() OVER (PARTITION BY ... ORDER BY s.valor DESC)** enumera as linhas por reservatório ordenadas pelo valor decrescente.
- ❑ **WHERE rn <= 5** mantém apenas as 5 primeiras de cada reservatório.
- ❑ Útil para listas “Top N”.
- ❑ **Observações sobre empates:** ROW_NUMBER() quebra empates arbitrariamente; se quiser retornar **todos** os empates no limite (por exemplo se houver vários registros com o mesmo valor que ocupe a 5ª posição) use RANK() e filtre RANK() <= 5.

Exercício 3: Reservatórios que nunca registraram Condutividade

- ❑ **Enunciado resumido:** listar reservatórios que não têm nenhuma medição do parâmetro Condutividade.

- ❑ -- Solução usando NOT EXISTS

```
SELECT r.nome AS reservatorio
FROM reservatorio r
WHERE NOT EXISTS (
    SELECT 1
    FROM serie_temporal s
    JOIN parametro p ON s.id_parametro = p.id_parametro
    WHERE s.id_reservatorio = r.id_reservatorio
    AND p.nome_parametro = 'Condutividade'
)
ORDER BY r.nome;
```

Explicação:

- Para cada reservatório *r*, a subconsulta procura qualquer medição de Condutividade com *s.id_reservatorio = r.id_reservatorio*.
- NOT EXISTS garante que nenhum registro exista, então o reservatório nunca registrou condutividade.

Exercício 3: Reservatórios que nunca registraram Condutividade

- ❑ **Alternativa com NOT IN (atenção a NULLs):**

```
SELECT r.nome
```

```
FROM reservatorio r
```

```
WHERE r.id_reservatorio NOT IN (
```

```
    SELECT s.id_reservatorio
```

```
    FROM serie_temporal s
```

```
    JOIN parametro p ON s.id_parametro = p.id_parametro
```

```
    WHERE p.nome_parametro = 'Condutividade'
```

```
);
```

- ❑ **Cuidado:** NOT IN pode comportar-se mal se subconsulta retornar NULLs; NOT EXISTS é mais seguro.

Exercício 4: Parâmetros cuja média global é maior que a média total de todos parâmetros

- ❑ **Enunciado resumido:** calcule a média por parâmetro; retorne os parâmetros cuja média é maior que a média **global** (média de todos os valores de serie_temporal, independentemente do parâmetro).

- ❑ -- Solução usando CTEs para clareza

```
WITH media_por_parametro AS (  
    SELECT p.nome_parametro,  
           AVG(s.valor) AS media_parametro  
    FROM serie_temporal s  
    JOIN parametro p ON s.id_parametro =  
    p.id_parametro  
    GROUP BY p.nome_parametro  
)
```

```
media_global AS (  
    SELECT AVG(valor) AS media_total  
    FROM serie_temporal  
)  
  
SELECT mpp.nome_parametro, mpp.media_parametro  
FROM media_por_parametro mpp, media_global mg  
WHERE mpp.media_parametro > mg.media_total  
ORDER BY mpp.media_parametro DESC;
```

Exercício 4: Parâmetros cuja média global é maior que a média total de todos parâmetros

- ❑ **Explicação:**
- ❑ **media_por_parametro** calcula a média por cada parâmetro.
- ❑ **media_global** calcula a média de **todas** as medições.
- ❑ A consulta final compara e lista apenas os parâmetros com média superior à média geral.
- ❑ **Observação:** se quiser o número de registros por parâmetro junto com a média, acrescente COUNT(*) no CTE.

Exercício 5: Para cada reservatório, exibir o parâmetro com maior valor registrado

- ❑ **Enunciado resumido:** para cada reservatório, indicar qual parâmetro atingiu o maior valor (e mostrar esse valor e a data). Tratar empates se necessário.
- ❑ **Opção A: retornar um parâmetro por reservatório (escolhe arbitrariamente um se houver empate) — ROW_NUMBER():**

Exercício 5: Para cada reservatório, exibir o parâmetro com maior valor registrado

- ❑ -- Versão: um registro por reservatório (ROW_NUMBER)

```
SELECT reservatorio, nome_parametro, valor, data_hora
FROM (
    SELECT r.id_reservatorio,
           r.nome AS reservatorio,
           p.nome_parametro,
           s.valor,
           s.data_hora,
           ROW_NUMBER() OVER (
               PARTITION BY r.id_reservatorio
               ORDER BY s.valor DESC
           ) AS rn
```

```
FROM serie_temporal s
    JOIN parametro p ON s.id_parametro =
p.id_parametro
    JOIN reservatorio r ON s.id_reservatorio =
r.id_reservatorio
) t
WHERE rn = 1
ORDER BY reservatorio;
```

Explicação:

- ROW_NUMBER() dá exatamente 1 linha por partição; ótimo quando quer apenas um representante.
- RANK() mantém empates com o mesmo rank (1 para os maiores), então retorna mais de uma linha em caso de empate — útil se quiser ver todos os parâmetros que atingiram o mesmo maior valor.

Exercício 5: Para cada reservatório, exibir o parâmetro com maior valor registrado

- ❑ **Alternativa com DISTINCT ON (Postgres):**

```
SELECT DISTINCT ON (r.id_reservatorio)
    r.nome AS reservatorio,
    p.nome_parametro,
    s.valor,
    s.data_hora
FROM serie_temporal s
JOIN parametro p ON s.id_parametro =
p.id_parametro
JOIN reservatorio r ON s.id_reservatorio =
r.id_reservatorio
ORDER BY r.id_reservatorio, s.valor DESC;
```

- ❑ *DISTINCT ON é conciso e retorna a primeira linha por id_reservatorio segundo a ordem definida (aqui, maior valor). Também retorna um só registro por reservatório (que pode ser arbitrário em empates).*

BANCO DE DADOS RELACIONAL

Stored Procedure

Objetivos da aula



Conteúdo

O que é Stored Procedure

Diferença SP x Function

Criar uma Stored Procedure

Executar com parâmetros

Aplicar no banco de exemplo

Aplicar no ABP

Objetivo

Entender para quê serve

Saber quando usar cada

Sintaxe e execução

Entrada e lógica

Biblioteca

clima_alerta — INPE/limnologia

Conceitos rápidos

O que é uma Stored Procedure?

Um bloco de código SQL armazenado no banco que executa ações quando chamado

Ela serve para:

- > automatizar tarefas repetitivas
- > organizar lógica de negócio no banco
- > garantir integridade das regras
- > melhorar performance (menos ida e volta ao BD)
- > permitir execução com um comando simples: CALL

Analogia para explicar:

Sem Stored Procedure = toda vez você escreve tudo do zero

Com Stored Procedure = você tem uma “função” salva e só chama ela

Diferença: Stored Procedure x Function

Stored Procedure	Function
Executa ações (inserir, atualizar, deletar)	Calcula e retorna valores
Pode não retornar nada	Sempre retorna algo
Usa CALL	Usa SELECT function()
Ideal para regras de negócio	Ideal para cálculos e validações

Exemplo simples (Banco Biblioteca)

❑ Cenário

De biblioteca: vamos registrar um novo livro automaticamente

❑ Criar tabela (se ainda não criou)

```
CREATE TABLE livro (
    id_livro SERIAL PRIMARY KEY,
    titulo VARCHAR(100),
    autor VARCHAR(100),
    ano INTEGER
);
```

❑ Stored Procedure simples: inserir livro

```
CREATE PROCEDURE inserir_livro(
    titulo_p VARCHAR,
    autor_p VARCHAR,
    ano_p INT
)
LANGUAGE SQL
AS $$
    INSERT INTO livro(titulo, autor, ano)
    VALUES (titulo_p, autor_p, ano_p);
$$;
```

Exemplo simples (Banco Biblioteca)

❑ Executar

CALL inserir_livro('Dom Casmurro', 'Machado de Assis', 1899);

CALL inserir_livro('Memórias Póstumas', 'Machado de Assis', 1881);

❑ Verificar

*SELECT * FROM livro;*

Aplicação no Projeto ABP (clima_alerta/limnologia)

- ❑ **Exemplo:** registrar uma medição de qualidade da água
- ❑ **Base (ajustar conforme seu esquema final)**

```
CREATE PROCEDURE inserir_medicao(  
    id_reservatorio_p INT,  
    id_parametro_p INT,  
    valor_p NUMERIC,  
    data_p TIMESTAMP  
)
```

LANGUAGE SQL

AS \$\$

```
INSERT INTO serie_temporal(  
    id_reservatorio, id_parametro, valor,  
    data_hora  
) VALUES (  
    id_reservatorio_p, id_parametro_p, valor_p,  
    data_p  
);  
$$;
```

- ❑ **Executar**

```
CALL inserir_medicao(1, 2, 7.4, '2025-10-10  
10:00:00');
```

Atividade Prática (Individual)

A) Biblioteca

- ❑ Criar SP para atualizar o autor de um livro
- ❑ Criar SP para excluir livro pelo id

B) Projeto ABP

- ❑ Criar SP para cadastrar reservatório
- ❑ Criar SP para cadastrar parâmetro ambiental
- ❑ Criar SP para registrar medição *(se ainda não fez)*

C) Bônus avançado

- ❑ Criar SP com validação que:
- ❑ Não permite valor negativo
- ❑ Mostra mensagem de erro (RAISE EXCEPTION)

Boas práticas

Prática

- Nome claro e padrão
- Receber parâmetros explícitos
- Documentar dentro da SP
- Validar parâmetros
- Usar transações quando necessário
- Evitar lógica de negócios excessiva
- Manter simples e modular
- Versão e histórico de alterações

Explicação

- sp_inserir_usuario, sp_registrar_medicao
- Evita depender de estado externo
- Comentários explicando o propósito
- Evita dados incorretos (ex.: valores negativos)
- BEGIN; ... COMMIT; em operações críticas
- Use SP para regras de dados, não regras de tela
- Uma SP — um propósito
- Criar padrão de documentação para alterações

Qual a vantagem de usar SP?

Benefício

- Performance
- Segurança
- Reutilização
- Consistência
- Atomicidade
- Automação
- Melhor organização

Por quê importa?

- Processa dentro do BD: menos tráfego
- Pode restringir acesso direto às tabelas
- Regras e operações complexas em único ponto
- Mesma regra sempre aplicada
- Garantem que operações de banco sejam seguras
- Operações repetitivas ficam automáticas
- Banco mais profissional e gerenciável

Stored Procedures transformam o banco em um agente ativo do sistema, não apenas um repositório de dados.

Desvantagens / limitações

Desvantagem

Manutenção mais difícil
Pode gerar dependência do SGBD
Versão e controle de código mais complexo
Excesso de lógica no BD pode confundir
Debug mais limitado

Impacto

Código espalhado entre backend e BD
Difícil migrar entre PostgreSQL / MySQL / etc
Requer disciplina com Git
SP deve complementar, não substituir Backend
SP não é tão fácil de testar quanto código TS/JS

*SPs são poderosas e se usado demais, vira bagunça.
Se usado certo, vira diferencial profissional.*

Quando usar Views?

Cenário	Usar SP?	Motivo
Rotinas repetitivas	Sim	Evita reescrever código
Rotinas críticas	Sim	Segurança e atomicidade
Regras de negócios sensíveis	Sim	Garantir integridade
Muita lógica no dado	Sim	Calcula dentro do banco, mais rápido
CRUD simples	Não	Pode ser excesso
Lógica de UI / Client	Não	Deve ficar no frontend
Validações complexas de app	Depende	Depende: às vezes melhor no backend

SP é usada quando a regra é do dado, não da tela.

Perguntas frequentes

- ❑ **“Professor, posso fazer tudo no backend e ignorar SP?”**
Pode, mas vai perder **segurança, performance e consistência**.
Bancos modernos não são caixa de dados — eles processam dados.
- ❑ **“Por que o PostgreSQL usa CALL, e não SELECT?”**
CALL = procedimento (ação)
SELECT = função (retorna valor)
No PostgreSQL moderno, a sintaxe segue o padrão SQL.

❑ **Boas Práticas**

Nome claro (sp_registrar_medicao)
Validar parâmetros
Documentar
Modularizar
Usar transação quando necessário

❑ **Vantagens**

Performance
Segurança
Reutilização
Consistência
Automação

❑ **Desvantagens**

Complexidade de manutenção
Dependência do SGBD
Controle de versão mais difícil

❑ **Use SP quando**

Regras de dado
Operações críticas
Rotinas repetitivas

SP = lógica **próxima do dado** -> profissionalismo e robustez

O que veremos na próxima aula

- ❑ **Stored Procedures**
- ❑ Tópicos:
 - ❑ Stored Procedure com regras de negócios
 - ❑ Boas práticas
 - ❑ Erros comuns
 - ❑ SP com IF, LOOP e validações
 - ❑ Exercícios de simulação real INPE



Referências Bibliográfica da Aula

Livros:

Elmasri & Navathe (2010). Sistemas de Banco de Dados.

Silberschatz et al. (2011). Sistemas de Banco de Dados.

Links úteis:

 [PostgreSQL Docs](#)

 [DBDiagram.io](#)

Bibliografia Básica

- ❑ DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro, Elsevier: Campus, 2004.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7 ed. São Paulo: Pearson, 2018.
- ❑ SILBERSCHATZ, A.; SUNDARSHAN, S.; KORTH, H. F. **Sistema de banco de dados**. Rio de Janeiro: Elsevier Brasil, 2016.

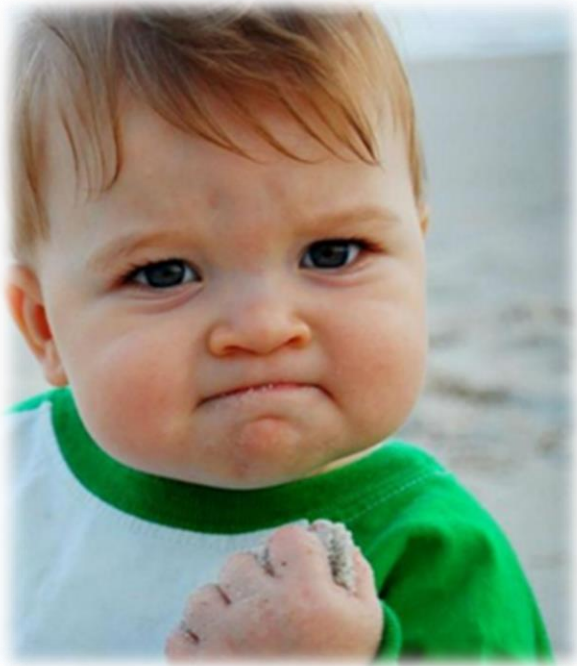
Bibliografia Complementar

- ❑ BEAULIEU, A. **Aprendendo SQL**. São Paulo: Novatec, 2010.
- ❑ GILLENSON, M. L. **Fundamentos de Sistemas de Gerência de Banco de Dados**. Rio de Janeiro: LTC, 2006.
- ❑ MACHADO, F. N. R. **Banco de Dados: Projeto e Implementação**. São Paulo: Érica, 2005.
- ❑ OTEY, M; OTEY, D. **Microsoft SQL Server 2005: Guia do Desenvolvedor**. Rio de Janeiro: Ciência Moderna, 2007.
- ❑ RAMAKRISHNAN, R.; GEHRKE, J. **Sistemas de Gerenciamento de Bancos de Dados**. 3 ed. Porto Alegre: Bookman, 2008.
- ❑ ROB, P; CORONEL, C. **Sistemas de Banco de Dados: Projeto, Implementação e Gerenciamento**. 8 ed. São Paulo: Cengage Learning, 2011.
- ❑ TEOREY, T; LIGHTSTONE, S; NADEAU, T. **Projeto e Modelagem de Bancos de Dados**. São Paulo: Campus, 2006.

Dúvidas?



Considerações Finais



**Professora:
Lucineide Pimenta**

Bom descanso à todos!

